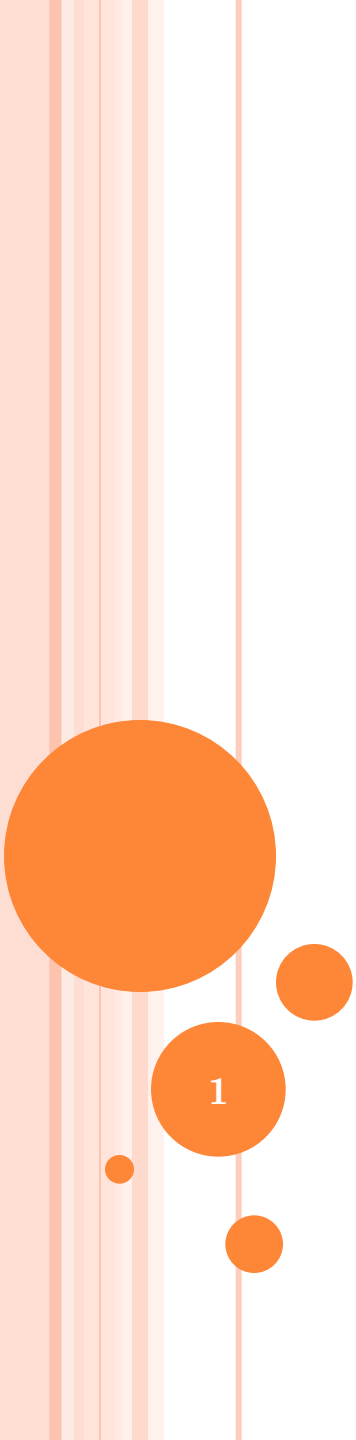




SISTEME DE OPERARE

Curs 8 și 9: Concurență și sincronizare

Kinga Márton

Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare

CONCURENȚĂ ȘI SINCRONIZARE

○ În cuprins

- Introducere
 - Concurență → necesitatea sincronizării
- Concepte fundamentale
 - Regiune critică
 - Excludere mutuală
- Implementarea regiunilor critice
 - Soluții software și hardware
- Probleme clasice de sincronizare



INTRODUCERE

- Gestiunea proceselor și threadurilor
 - Sarcina centrală a SO
 - **Multiprogramare**
 - Gestiunea proceselor multiple în cadrul unui sistem monoprocesor
 - **Multiprocesare**
 - Gestiunea proceselor multiple în cadrul sistemelor cu mai multe procesoare
 - **Procesare distribuită**
 - Gestiunea proceselor multiple care se execută pe mai multe sisteme distribuite (ex. clustere)
- Concept fundamental : **CONCURENȚA**
 - Gestiunea proceselor active
 - Comunicare între procese / interacțiunea
 - Alocarea și partajarea resurselor (memorie, fișiere, acces I/E)
 - Sincronizarea activităților
 - Rezultatul să fie independent de diferența dintre vitezele relative ale proceselor concurente
 - Alocarea timpului procesor la procese

CONCURENȚA

○ Apare în **trei contexte** diferite

- Mai multe aplicații

- Multiprogramarea – partajarea dinamică a timpului de procesare între mai multe aplicații active

- Structura aplicațiilor

- mai multe procese (threaduri) concurente

- Structura SO

- implementat sub forma unui set de procese și threaduri

○ Excludere mutuală

- Cerința fundamentală prin care se oferă suport pentru procese concurente

○ **Mecanisme** pt asigurarea excluderii mutuale

- Mecanisme software
- Mecanisme hardware
- Soluții furnizate de SO sau de compilator
 - Semafoare, lacăte, variabile condiționale

PRINCIPIUL CONCURENȚEI

○ Execuție **intercalată** și **suprapusă**

- **Intercalarea** în timp a execuției oferă iluzia execuției simultane pe sisteme monoprosesor

- Introduce o încărcare suplimentară (schimbările de context) dar
- Oferă eficiență sporită în procesare și în structurarea programului

- În cazul sistemelor multiprosesor este posibilă nu doar intercalarea dar și **suprapunerea** execuțiilor (paralelism real)

- Ambele sunt exemple de execuție concurentă
- Ambele prezintă probleme similare
 - Date de concurență

PRINCIPIUL CONCURENȚEI

- Viteza relativă de execuție a proceselor nu este predictibilă
 - Rezultă din caracteristică de bază a multiprogramării
 - Depinde de activitățile celorlalte procese
 - De modul de gestiune a întreruperilor de către SO
 - De politica de planificare folosită de SO
- Execuția concurentă necontrolată poate duce la rezultate imprecizabile și stări inconsistente ale resurselor partajate
 - Ex. două procese/threaduri partajează o valoare și ambele procese citesc și scriu această valoare -> ordinea de execuție a citirilor și scrierilor este critică
 - Dificultatea alocării optime a resurselor de către SO (fără interblocare)

CONCEPTE FUNDAMENTALE

- Operații atomice (primitive)
- Condiție de cursă (race condition)
- Regiune critică
- Excludere mutuală
- Interblocare (deadlock)
- Înfometare (starvation)

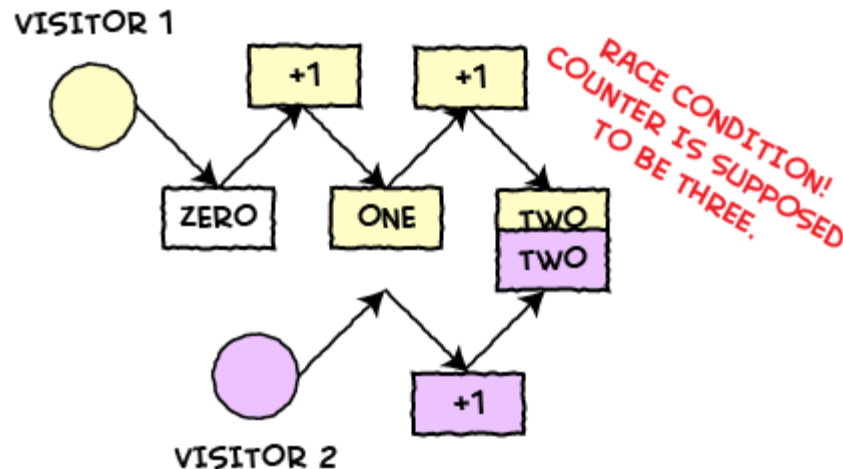
CONCEPTE FUNDAMENTALE

○ Operații atomice

- Acțiune implementată ca o secvență de instrucțiuni **indivizibile**
 - Nici un alt proces **nu poate vedea o stare intermediară**
 - Nici un alt proces **nu poate întrerupe execuția secvenței**
- Principiu: Totul sau nimic
 - Este garantată execuția secvenței în totalitate sau deloc
- Atomicitatea garantează izolarea de procesele concurente
- Necesitatea sincronizării se datorează faptului că operațiile asupra resurselor partajate nu sunt atomice
 - Pot fi întrerupte fără finalizarea actualizării resurselor partajate

CONCEPTE FUNDAMENTALE

- **Condiție de cursă** (race condition)
 - Situație în care mai multe procese (threaduri) citesc și scriu o dată partajată și rezultatul final depinde de ordinea relativă a execuțiilor
- **Resursă critică**
 - Resursă accesată de mai multe procese în mod concurent
- Un exemplu simplu – incrementarea concurentă



CONCEPTE FUNDAMENTALE

- Condiție de cursă (race condition)
 - Un exemplu simplu – incrementarea concurentă

Resursă critică	Thread 1	Thread 2
<code>int x=0;</code>	<code>x++;</code>	<code>x++;</code>

- Implementare incrementării

```
registru1 = x
registru1 = registru1 + 1
x = registru1
```

CONCEPTE FUNDAMENTALE

- Rezultatul ar putea fi

Thread 1	Thread 2	r1	r2	x
<code>r1 = x</code>		0		0
<code>r1 = r1+1</code>		1		0
<code>x = r1</code>		1		1
	<code>r2 = x</code>		1	1
	<code>r2 = r2+1</code>		2	1
	<code>x = r2</code>		2	2

CONCEPTE FUNDAMENTALE

- Rezultatul poate fi și acesta
 - Planificare preemptivă

Thread 1	Thread 2	r1	r2	x
<code>r1 = x</code>		0		0
<code>r1 = r1+1</code>		1		0
	<code>r2 = x</code>	1	0	0
	<code>r2 = r2+1</code>	1	1	0
<code>x = r1</code>		1	1	1
	<code>x = r2</code>	1	1	1

CONCEPTE FUNDAMENTALE

○ Regiune critică (Critical section)

- Constă dintr-un număr de instrucțiuni consecutive în care se accesează resurse critice în mod concurent

Proces {

. . .

Acces la resurse partajate;

. . .

}

CONCEPTE FUNDAMENTALE

○ Regiune critică (Critical section)

○ Ideea de bază

- Accesul la resurse partajate se face doar în cadrul regiunilor critice
- Fiecare proces trebuie să ceară permisiune să intre în regiune
 - Poate implica așteptare
- Asociem câte un lacăt la fiecare resursă partajată

Proces {

. . .

INTRĂ ÎN REGIUNEA CRITICĂ

Acces la resurse partajate;

PĂRĂSEȘTE REGIUNEA CRITICĂ

. . .

}

CONCEPTE FUNDAMENTALE

○ Regiune Critică (Critical section)

- Implementarea secțiunii critice trebuie să asigure
 - **Corectitudine**
 - Pentru un număr k dat, doar k procese /threaduri pot fi în regiunea critică în orice moment (în general $k=1$ – înseamnă excludere mutuală)
 - **Controlul concurenței**
 - Să permită concurență maximă în timp ce menține corectitudinea
 - **Eficiență**
 - Intrarea și ieșirea în /din regiunea critică trebuie să se facă rapid
 - **Flexibilitate**
 - Să înglobeze cât mai puține restricții
- Cerințe
 - **Excludere mutuală**
 - **Progres**
 - **Așteptare limitată**

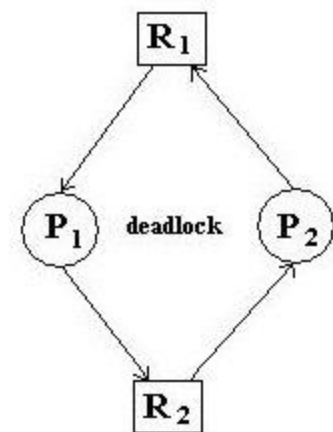
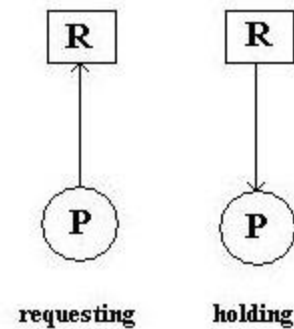
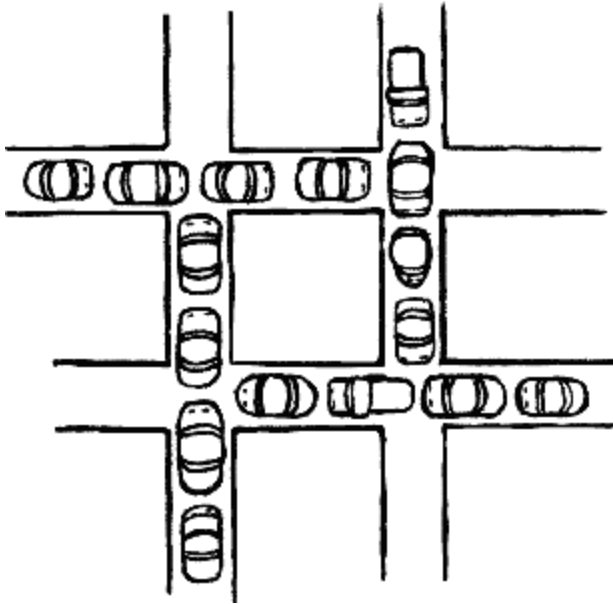
REGIUNE CRITICĂ – CERINȚE

○ Excludere mutuală

- În orice moment doar **un singur proces** poate fi în regiunea critică
 - Cât timp un proces este în regiunea critică, nici un alt proces nu poate intra
- Asigurarea excluderii mutuale poate introduce două probleme adiționale
 - **Interblocarea** (deadlock)
 - Fiecare proces deține o resursă pentru care așteaptă un alt proces
 - Două procese așteaptă reciproc comunicare din partea celuilalt
 - **Înfometarea** (starvation)
 - Planificare amânată un timp nedeterminat
 - Unui proces i se refuză accesul la o resursă, chiar dacă nu este o situație de deadlock

CONCETTO FUNDAMENTALE

- Interblocare - deadlock



REGIUNE CRITICĂ – CERINȚE

○ Excludere mutuală

- Cât timp un proces este în regiunea critică, nici un alt proces nu poate intra

○ Progres (evitarea interblocării)

- Dacă nici un proces nu este în regiunea critică și mai multe procese așteaptă să intre în regiunile lor critice, atunci intrarea nu poate fi amânată un timp nedefinit
 - Intrare fără amânare dacă regiunea critică este liberă
 - Procese care rulează în afara regiunii critice nu pot bloca alte procese

○ Așteptare limitată (evitarea înfometării)

- Există o limită asupra numărului de procese care pot intra în regiunea critică după ce un alt proces a cerut acces și până la acordarea permisiunii
 - Un proces rămâne în regiunea critică doar pentru un timp limitat
 - Presupunem că fiecare proces se execută cu o *viteză nenulă*
 - Nu putem presupune nimic legat de vitezele relative de execuție ale proceselor concurente

REGIUNE CRITICĂ – CERINȚE

- Dacă sunt permise mai multe procese în regiunea critică atunci excluderea mutuală se transformă în
- Proprietatea de siguranță
 - Pot intra în regiunea critică doar un număr dat k de procese

CONCEPTE FUNDAMENTALE



CONCEPTE FUNDAMENTALE



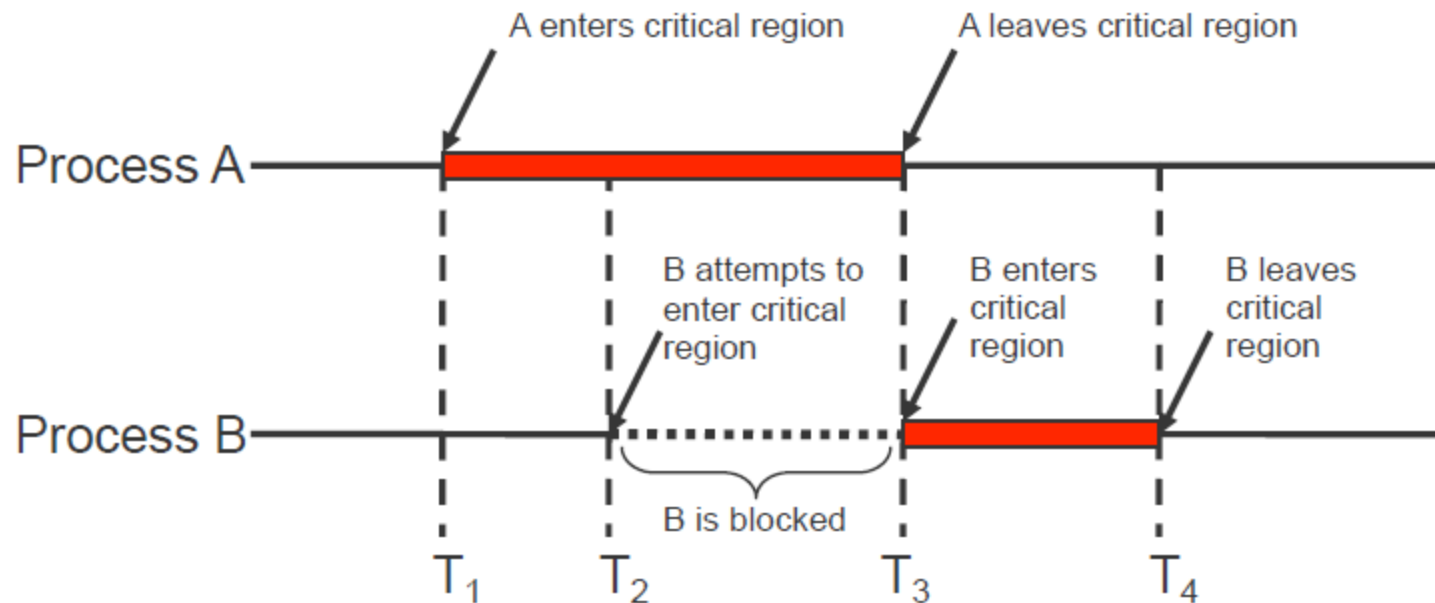
CONCEPTE FUNDAMENTALE



CONCEPTE FUNDAMENTALE



EXCLUDERE MUTUALĂ FOLOSIND REGIUNI CRITICE



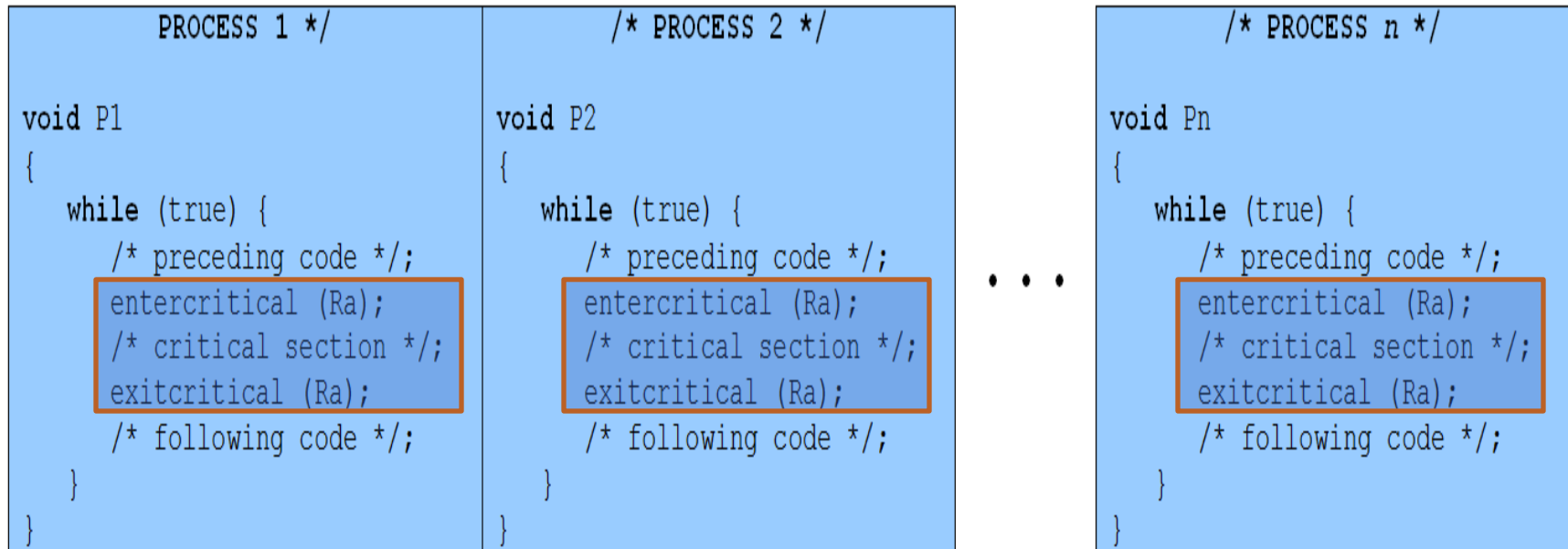
EXCLUDERE MUTUALĂ

○ Ilustrare în termeni abstracti

- N procese concurente – fiecare include
 - **Regiune critică** care operează pe aceeași resursă Ra
 - **Cod adițional** înainte și după regiunea critică
 - Unde nu se acesează resurse partajate

○ Vom studia mecanisme specifice

- pentru implementarea funcțiilor *entercritical()* și *exitcritical()*



PRINCIPIUL CONCURENȚEI

○ Interacțiunea dintre procese / threaduri

- **Competiție vs. Cooperare**

- **Tabel: model simplu** (în realitate procesele pot prezenta atât aspecte de cooperare cât și de competiție în același timp)

Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	• Results of one process independent of the action of others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) • Starvation

PRINCIPIUL CONCURENȚEI

- Explicații pt tabelul precedent
- In primele două cazuri – **competiție** și **cooperare prin partajare**
 - fiecare proces are propriul mediu izolat – care nu include celelalte procese
 - Interacțiunea dintre procese este indirectă
- In ambele cazuri există partajare:
 - In cazul competiției procesele partajează resurse fără să fie conștiente de existența altor procese
 - In cazul al doilea, procesele partajează valori, și deși procesele nu sunt explicit conștiente de existența altor procese, sunt conștiente de faptul că trebuie să mențină integritatea datelor (coerența, consistența datelor)
- Când procesele **cooperează prin comunicare** – multiplele procese participă la un efort comun care leagă toate procesele
 - Comunicarea oferă o cale de a se sincroniza, sau de a coordona activitățile diverse
- De regula comunicarea se realizează prin schimbarea de mesaje
 - deci nu este nimic partajat între procese – astfel excluderea mutuală nu este necesară, dar poate apărea interblocarea și infometarea
- **Interblocare**: două procese P1 și P2 – ambele procese așteaptă comunicarea de la celălalt
- **Infometare**: considerăm procesele P1, P2, P3.
 - P1 comunică atât cu P2 cât și cu P3
 - Dacă P1 și P2 comunică repetat, și P3 este blocat așteptând comunicare de la P1 → P3 este infometat (nu e interblocare, deoarece P1 este activ).

STUDIU DE CAZ

- Problema cumpărării laptelui – “Too much milk problem”

	<u>Person A</u>	<u>Person B</u>
3:00	Look in fridge. Out of milk.	-
3:05	Leave for store.	-
3:10	Arrive at store.	Look in fridge. Out of milk.
3:15	Buy milk.	Leave for store.
3:20	Arrive home, put milk in fridge.	Arrive at store.
3:25	-	Buy milk.
3:30	-	Arrive home, put milk in fridge

- Fără cooperare – prea mult lapte
- Proprietate de corectitudine impune ca
 - Doar o singură persoană să cumpere lapte
 - Siguranță: "nothing bad happens"
 - Cineva va cumpăra lapte când e nevoie
 - Progres: "something good eventually happens"

STUDIU DE CAZ

○ Soluția nr. 1: Variabila lacăt (lock)

- Persoana care merge să cumpere lapte lasă un mesaj pe frigider
 - Cu funcția similară unui lacăt
 - Lasă mesaj : pune lacătul
 - Îndepărtează mesaj : deschide lacătul

```
person ()
```

```
{  
    if (noMilk) {  
        if (noNote) {  
            leave Note;  
            buy milk;  
            remove note;  
        }  
    }  
}
```

○ Problemă – soluția eșuează ocazional (mai rău decât înainte)

- Ambele procese execută același cod
- Pot fi simultan în regiunea critică
 - schimbare de context după testarea dacă există mesaj dar înainte să se pună mesajul

STUDIU DE CAZ

○ Soluția nr. 2: Pre-blocare

- Anunțarea intenției înaintea testării contextului
 - Pune mesajul etichetat înainte să se uite în frigider

person_A()

```
{  
  leave note_A; 1  
  if (noNote_B) 4  
    if (noMilk)  
      buy milk;  
  remove note_A; 5  
}
```



person_B()

```
{  
  leave note_B; 2  
  if (noNote_A) 3  
    if (noMilk)  
      buy milk;  
  remove note_B; 6  
}
```



○ Se asigură excluderea mutuală

- Doar un singur proces este în regiunea critică

○ Problema: înfometarea

- Este posibil ca nici un proces să nu intre în regiunea critică
- Așteptare pentru o perioadă nedefinită

STUDIU DE CAZ

○ Soluția nr. 3: Pre-blocare cu evitarea înfometării

- Dacă amândoi lasă bilețul, atunci unul trebuie să se asigure că se cumpără laptele – alegem A

person_A()

```
{
    leave note_A;
    if (noNote_B)
        if (noMilk)
            buy milk;
    else {
        while (note_B)
            doNothing;
        if (noMilk)
            buy milk;
    }
}
remove note_A;
```

person_B()

```
{
    leave note_B;
    if (noNote_A)
        if (noMilk)
            buy milk;
    remove note_B;
}
```

STUDIU DE CAZ

○ Soluția nr. 3: Pre-blocare cu evitarea înfometării

- Dacă amândoi lasă bilețul, atunci unul trebuie să se asigure că se cumpără laptele – alegem A

person_A()

```
{
    leave note_A;
    if (noNote_B)
        if (noMilk)
            buy milk;
    else {
        while (note_B)
            doNothing;
        if (noMilk)
            buy milk;
    }
}
remove note_A;
```

person_B()

```
{
    leave note_B;
    if (noNote_A)
        if (noMilk)
            buy milk;
    remove note_B;
}
```

○ Dezavantaje

- A trebuie să aștepte cât timp B este la cumpărături
- A așteaptă consumând resurse (**busy-waiting**)

STUDIU DE CAZ

○ Soluția nr. 4: Alternare strictă

- Impunerea unei ordini precise în care procesele își desfășoară activitatea
 - Persoana 1, persoana 2, ..., persoana n, persoana 1,.... etc.

```
int turn = 0;
int noPers = N;

person(int id) {
    // wait until my turn
    while (turn != id) { //do nothing }
    // it is my turn
    if (no milk)
        buy milk;
    // transfer the turn
    turn = (turn + 1)%noPers;
}
```

○ Problemă

- Impunerea ordinii -> viteza de execuție a fiecărui proces este redusă la viteza celui mai lent
- Dacă un proces se blochează atunci și celelalte sunt blocate
- Așteptare ocupată - Busy waiting

STUDIU DE CAZ

○ Concluzii

- Sincronizarea este dificilă (dacă este posibilă) prin **mecanisme software**
 - furnizate de **modul utilizator**
 - Unde procesele sunt responsabile pentru sincronizare – fără suport din partea SO sau a limbajului
- Ce se poate face?
 - Folosirea **mecanismelor hardware** pentru eliminarea aşteptării ocupate (busy waiting)
 - Definirea unor **abstracţii de nivel înalt** care să simplifice programarea concurentă
 - cu **suport din partea SO** sau compilator

SUPOORT HARDWARE PT SINCRONIZARE

- Sistemele pot oferi suport hardware pentru implementarea regiunii critice
 - Dezactivarea întreruperilor
 - Instrucțiuni mașină atomice
 - Testează & setează
 - Compară & interschimbă

SUPORT HARDWARE PT SINCRONIZARE

○ Dezactivarea întreruperilor

○ Funcționează doar pe sisteme monoprocesor

- Codul care rulează va fi executat fără preempțiune
 - Regiune atomică
- Garantează excluderea mutuală

○ Mod de folosire

- Fiecare proces
 - **dezactivează întreruperile** chiar înainte de a intra în regiunea critică
 - **Reactivează întreruperile** imediat după ieșirea din regiunea critică

○ Dezavantaj

- Pe perioada execuției regiunii critice procesul utilizator are control asupra întreruperilor
 - Dacă procesul se termină în regiunea critică -> **blochează sistemul**
- Eficiența execuției este semnificativ degradată
 - **Limitare în intercalarea execuțiilor**
- Nu este potrivit pt sisteme multiprocesor

INSTRUCȚIUNI MAȘINĂ SPECIALE

- Pe sisteme multiprocesor

unde mai multe procesoare partajează memoria principală comună

- Nu există un mecanism de întrerupere între procesoare pe care să se bazeze excluderea mutuală
 - Procesoarele se comportă independent, în relație de parteneriat
- La nivel hardware – accesul la o locație de memorie exclude orice alt acces la aceeași locație
 - De aici inspirația de a introduce **instrucțiunile atomice care să efectueze două acțiuni în mod indivizibil** (atomic)
 - Citirea și scrierea
a unei locații de memorie într-un singur IFC (Instruction Fetch Cycle)
 - Excludere mutuală chiar și între mai multe procesoare
- Compară și schimbă (Compare & swap)
- Interschimbă (Exchange)

INSTRUCȚIUNI MAȘINĂ SPECIALE

○ Compare & swap

- Compara valoarea de la o locație de memorie (*word) cu o valoare de test (testval)
 - Dacă au valori identice -> valoarea veche (*word) este înlocuită cu valoarea nouă (newval)
 - Altfel valoarea de la aceea locație rămâne neschimbată
- Se returnează valoarea veche
- Toate acestea în mod atomic

```
int compare_and_swap (int *word, int testval, int newval)
{
    int oldval;
    oldval = *word
    if (oldval == testval) *word = newval;
    return oldval;
}
```

INSTRUCȚIUNI MAȘINĂ SPECIALE

○ Compare & swap

- **Exemplu** de implementare a excluderii mutuale

- Variabila partajata `bolt` se initializează la 0
- Singurul proces care poate intra în regiunea critică este acela care găsește `bolt` cu valoarea 0
- Celelalte procese sunt blocate în așteptare ocupată (busy waiting)
 - Procesul nu poate face nimic altceva decât să testeze condiția de intrare în regiunea critică

```
/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /* do nothing */;
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ... , P(n));
}
```

INSTRUCȚIUNI MAȘINĂ ATOMICE

○ Concluzii

• Avantaje

- Aplicabil la orice număr de procese atât pe sisteme monoprocesor cât și multiprocesor
- Oferă suport pentru implementarea mai multor regiuni critice – fiecare cu variabila proprie

• Dezavantaje

- Problema așteptării ocupate
 - Un proces care așteaptă acces la regiune critică va continua să consume timp de procesare
- Problema interblocării
 - Ex. P1 intră în reg. critică,
 - dar este întrerupt pentru P2 - proces cu prioritate mai mare;
 - P2 încearcă să intre în reg critică dar este blocat în așteptare ocupată;
 - P1 nu se va putea executa deoarece are prioritate mai joasă...
- Problema înfometării
 - La părăsirea reg. critice dacă mai multe procese așteaptă intrarea selectarea unuia este arbitrară

• Avem nevoie de alte mecanisme...

MECANISME DE SINCRONIZARE

- Semafoare
- Lacăte (mutex)
- Variabile de condiție

SEMAFOARE

- Mecanisme de sincronizare care nu necesită așteptare ocupată
 - Evită așteptarea ocupată prin **suspendare**
- **Variabilă întreagă** pe care sunt definite trei operații
 - Nu se poate manipula sau verifica valoarea semaforului decât prin cele trei operații definite
 - Semaphore s
- 1. Poate fi **inițializat** (valoare întreagă)
- 2. $\text{semWait}(s)$ **decrementează** valoarea semaforului s
 - La intrarea în regiunea critică
 - Se mai notează și cu P (proberen - test), wait, down
- 3. $\text{semSignal}(s)$ **incrementează** valoarea semaforului
 - La ieșirea din regiunea critică
 - Se mai notează și cu V (verhogen - increase), signal, up
- Operațiile de decrementare și incrementare sunt **atomice**

SEMAFOARE

- Operațiile de decrementare și incrementare **trebuie să fie atomice**
 - Cum implementăm acesta?
 - Nu tocmai această problemă vroiam să evităm?
 - Ne reîntoarcem la sincronizare bazată pe **compare&swap** ?
 - Așteptare ocupată!!!
 - Deci cu ce sunt semafoarele mai bune decât mecanismele hardware?
- Concluzii
 - Semafoarele implică așteptare ocupată
 - dar **numai pe durata efectuării operației semWait (sau P)**
 - La compare&swap aveam busy waiting pe toată durata regiunii critice

SEMAFOARE - IMPLEMENTARE

○ Valoare întreagă

- Trei operații asupra valorii
 - **Inițializare**
 - **Decrementare**
 - La intrarea în regiunea critică
 - **Incrementare**
 - La ieșirea din regiunea critică

○ Coada de așteptare

- Fiecare semafor are o coadă de așteptare asociată
- Două operații asupra cozii
 - **Adăugare**
 - Procesul care se blochează pe operația semWait(s) este pus în coada de așteptare a semaforului s
 - **Eliminare**
 - Procesul deblocat (trezit) este scos din coada de așteptare și plasat în coada proceselor Ready – pregătit de execuție

SEMAFOARE

Stallings pg. 215

```
struct semaphore {
    int count;
    queueType queue;
};
void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

Figure 5.3 A Definition of Semaphore Primitives

SEMAFOARE - EXEMPLU

Stallings pg. 219

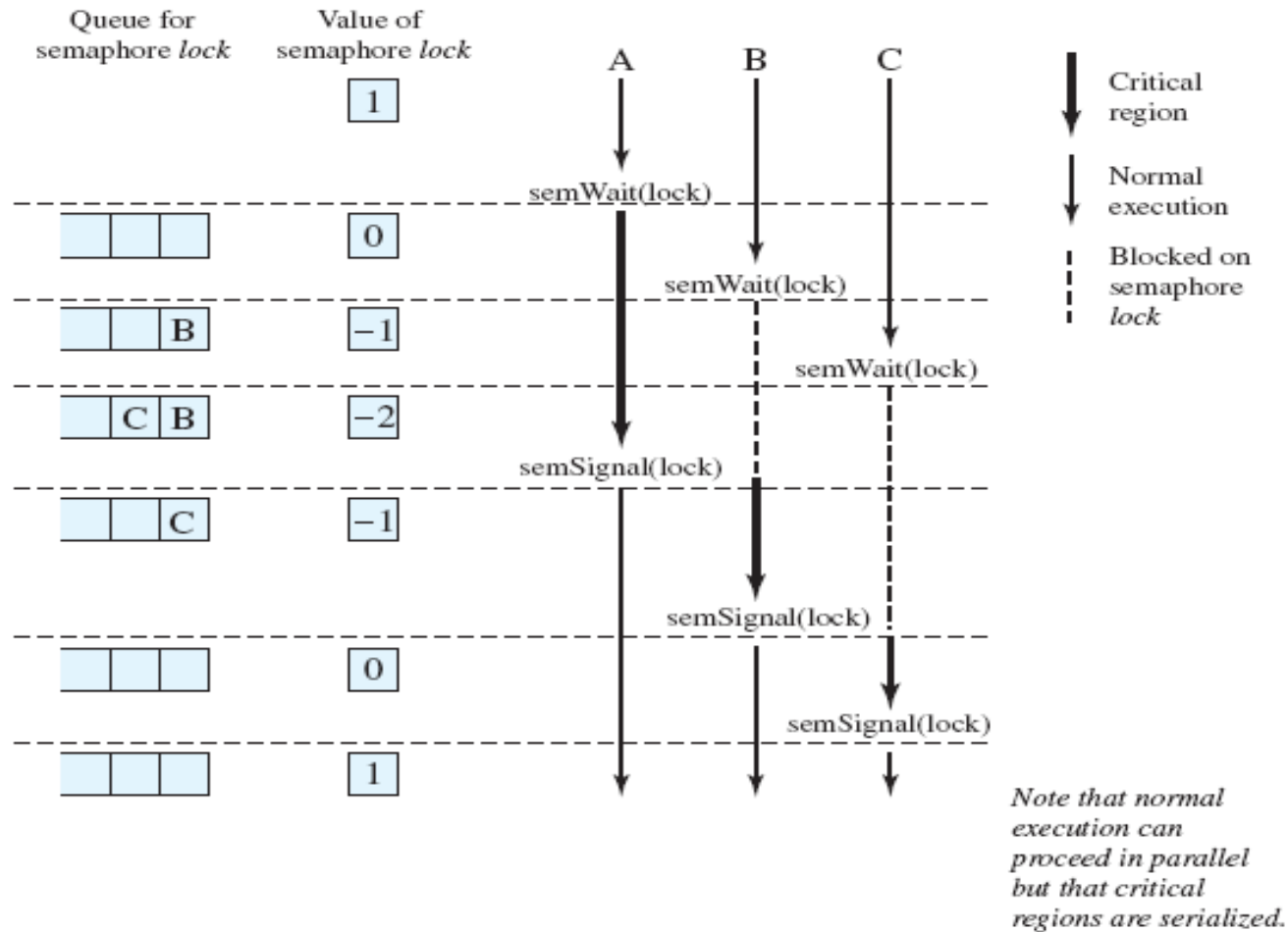


Figure 5.7 Processes Accessing Shared Data Protected by a Semaphore

SEMAFOARE

- Nu putem ști dinainte **dacă** un proces care decrementează semaforul **se va bloca** sau nu
- $V()$ trezește un proces care așteaptă, dar nu putem ști **care** proces **va continua** imediat execuția – fiind concurente pe un sistem monoprocesor
- Nu putem ști **dacă** un alt proces **așteaptă** – deci la V numărul proceselor deblocate poate fi 0 sau 1

SEMAFOARE CU CONTOR ȘI SEMAFOARE BINARE

○ Semafoare cu contor

- Valoarea poate fi dintr-un domeniu nerestricționat
- Permite unui număr limitat de procese / threaduri să execute concurent o secțiune de cod

○ Semafoare binare

- Cunoscut și sub numele de lacăt pentru **excludere mutuală** (mutex lock)
- Valoarea poate fi doar 0 sau 1
- Permite doar unui singur proces / thread să execute regiunea critică

SEMAFOARE BINARE

Stallings pg. 216

```
struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};
void semWaitB(binary_semaphore s)
{
    if (s.value == one)
        s.value = zero;
    else {
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignalB(semaphore s)
{
    if (s.queue is empty())
        s.value = one;
    else {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

Figure 5.4 A Definition of Binary Semaphore Primitives

EXCLUDERE MUTUALĂ FOLOSIND SEMAFOARE

Stallings pg. 218

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), . . . , P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

INTERBLOCARE ȘI ÎNFOMETARE

○ Interblocare

- Două (sau mai multe) procese așteaptă un eveniment care poate fi produs doar de celălalt (unul din celelalte) proces(e) blocat(e)
- S și Q sunt două semafoare inițializate la 1

P_0
wait (S);
wait (Q);

.
. .
.

signal (S);
signal (Q);

P_1
wait (Q);
wait (S);

.
. .
.

signal (Q);
signal (S);

○ Înfometare

- Este posibil ca un proces să nu fie niciodată scos din coada de așteptare a semaforului în care este suspendat
- Soluție: protocol de moștenire a priorității, random boosting

LACĂTE

- Mecanism pentru **excludere mutuală**
 - Similar cu semaforul binar
- Două stări distincte
 - **Deblocat** (eliberat, liber) – unlocked
 - Permite acces în regiunea critică
 - **Blocat** (ocupat) – locked
 - Nu permite acces în regiunea critică
- Operații atomice
 - Blocare – **Lock**
 - Apelat chiar înainte de intrarea în regiunea critică
 - Doar un proces poate bloca lacătul, chiar dacă mai multe încercă simultan
 - Deblocare – **Unlock**
 - Apelat imediat după terminarea regiunii critice
 - Doar procesul care a blocat lacătul poate să îl deblocheze

SĂ NE REAMINTIM...

○ Sincronizarea

- Se referă la o relație de ordine dintre evenimente
 - Oricâte evenimente
 - Orice relație de ordine
 - Un eveniment precedă alt eveniment
 - Un eveniment se desfășoară în timp ce alt eveniment este pe rol
 - Etc.
- Ex.
 - Evenimentul A trebuie să preceadă evenimentul B
 - Evenimentul A să nu se realizeze în același timp cu evenimentul B

○ Concurența

- Execuție intercalată
- Execuție suprapusă
- Aceleași probleme de sincronizare

SĂ NE REAMINTIM...

○ Viteza relativă de execuție

- În cadrul unui thread cunoaștem ordinea în care se execută operațiile
- ÎNSĂ - Nu cunoaștem viteza relativă de execuție a operațiilor din threaduri diferite

- Două evenimente sunt concurente dacă nu putem prezice – văzând doar codul sursă – care se va efectua primul - și pe ce cale de intercalare/suprapunere

- Și ordinea se poate schimba de la o rulare la alta
- --> nedeterminism
 - Face ca programele concurente să fie greu de verificat prin debug – testarea nu este suficientă!
 - Problemele trebuie evitate printr-o programare atentă și tratarea condițiilor de cursă prin mecanisme de sincronizare adecvate

SĂ NE REAMINTIM...

○ Resurse partajate

- Pentru asigurarea consistenței – de cele mai multe ori operația de actualizare trebuie efectuată în excludere mutuală
 - Excluderea mutuală garantează că resursa este accesată doar de un singur thread/proces în orice moment
- Constrângere: citirea valorii să se efectueze doar după terminarea scrierii / actualizării resursei
 - Problemă de serializare

SĂ NE REAMINTIM...

○ Resurse partajate

○ Scrieri concurente

- Ce valoare se va afișa? Depinde de ordinea de execuție a instrucțiunilor

- Ex. $a1 < a2 < b1$ → se afișează 5 dar valoarea finală este 7
- Ce cale de execuție va duce la

- Afișarea valorii 5 și valoare finală 5
- Afișarea valorii 7 și valoare finală 7
- Afișarea valorii 7 și valoare finală 5 – este posibilă ?

Ce căi sunt posibile –
cu ce efecte posibile?

- În cazul programelor concurente astfel de întrebări apar frecvent

○ Actualizări concurente

- Operațiile nu sunt atomice →



- Poate apare calea de execuție care duce la inconsistență:

$$a1 < b1 < b2 < a2$$

ȘABLOANE DE SINCRONIZARE

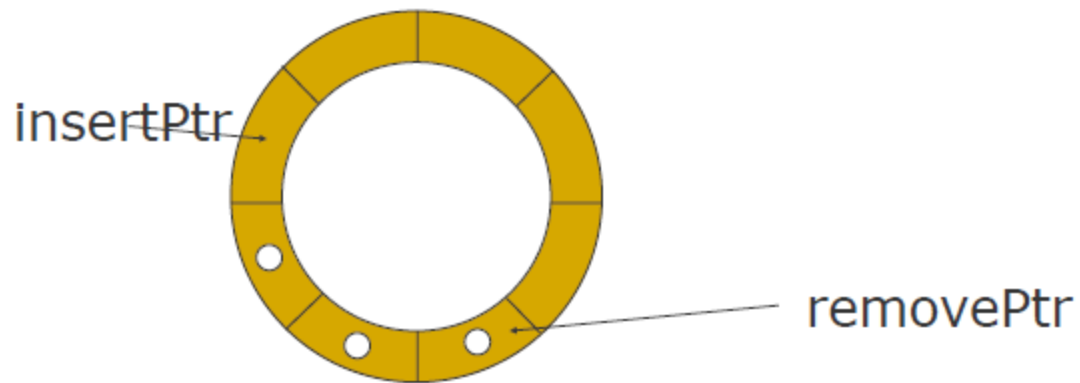
CAPITOLUL 3 DIN THE LITTLE BOOK OF SEMAPHORES

PROBLEME CLASICE DE SINCRONIZARE

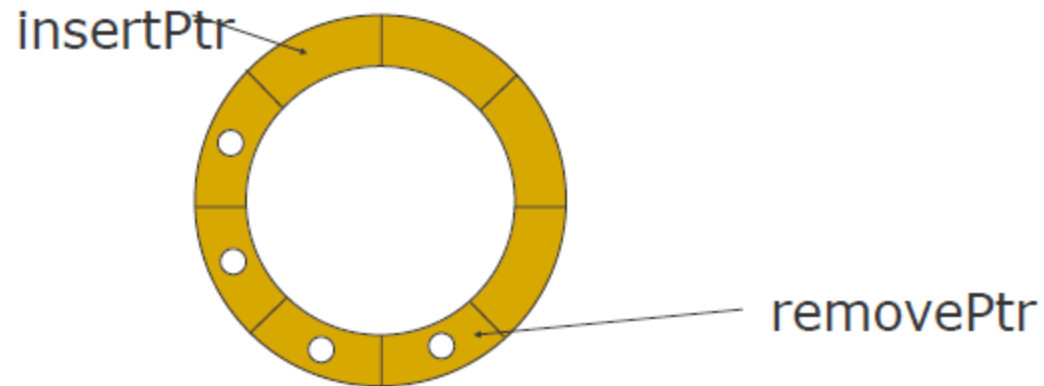
PROBLEMA PRODUCĂTOR – CONSUMATOR

- Două tipuri de procese
 - **Producător**: produce mesaje (date)
 - **Consumator**: consumă mesaje (date)
- Buffer cu capacitate limitată
 - Implementarea bufferului
 - buffer circular cu pointer pt inserare și extragere

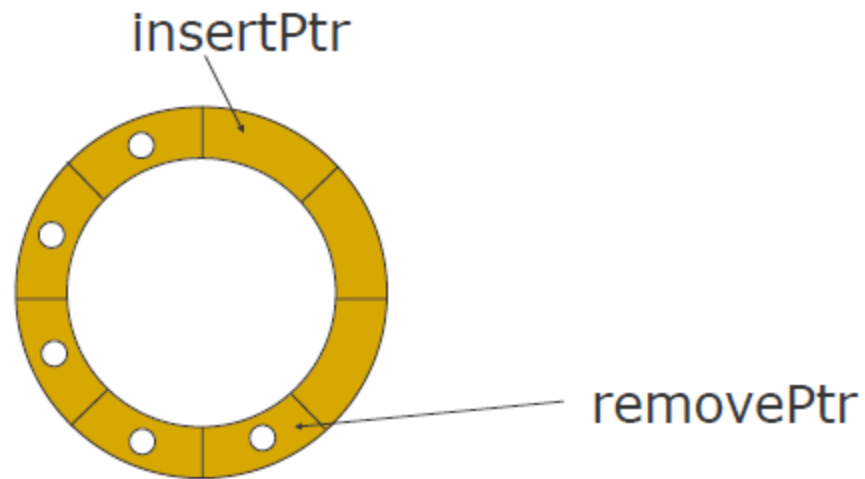
PROBLEMA PRODUCĂTOR – CONSUMATOR



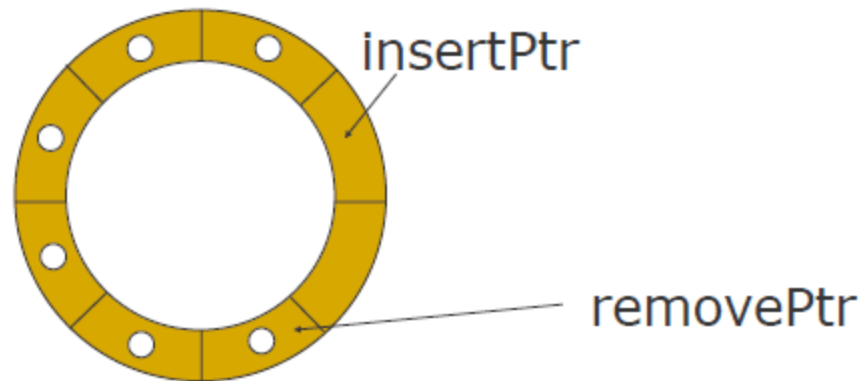
PROBLEMA PRODUCĂTOR – CONSUMATOR



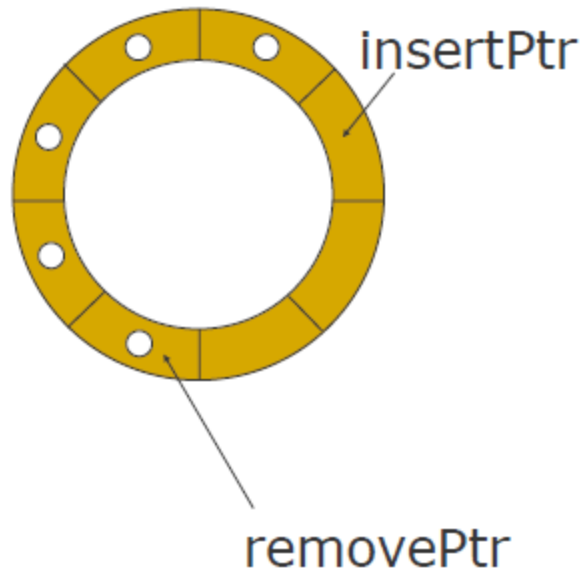
PROBLEMA PRODUCĂTOR – CONSUMATOR



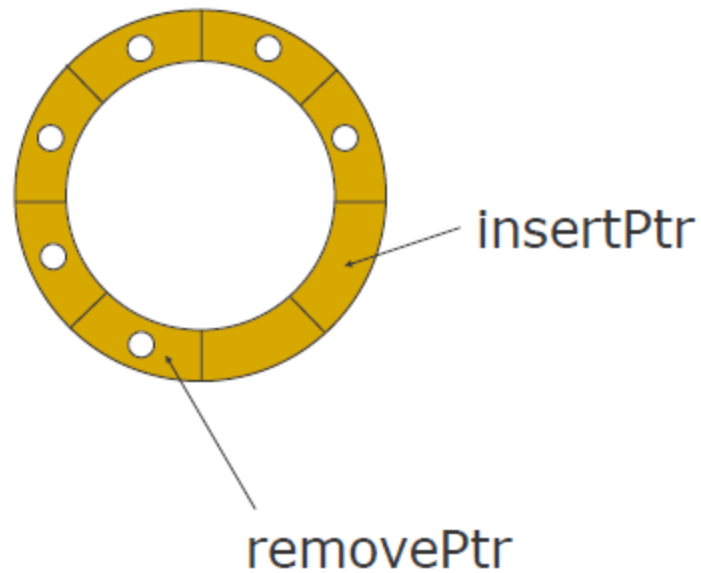
PROBLEMA PRODUCĂTOR – CONSUMATOR



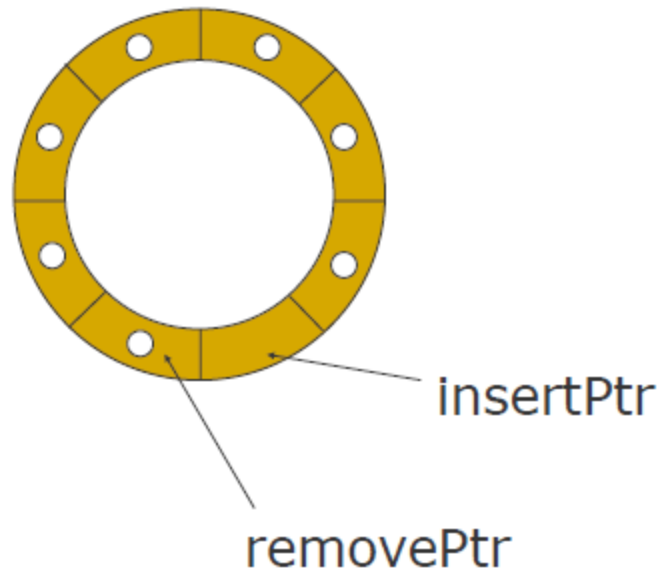
PROBLEMA PRODUCĂTOR – CONSUMATOR



PROBLEMA PRODUCĂTOR – CONSUMATOR

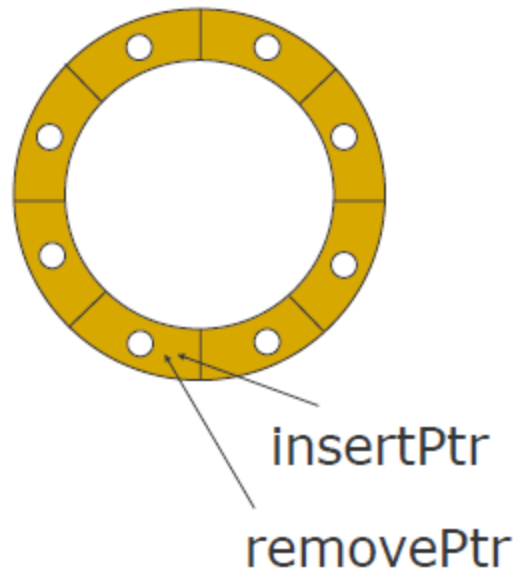


PROBLEMA PRODUCĂTOR – CONSUMATOR

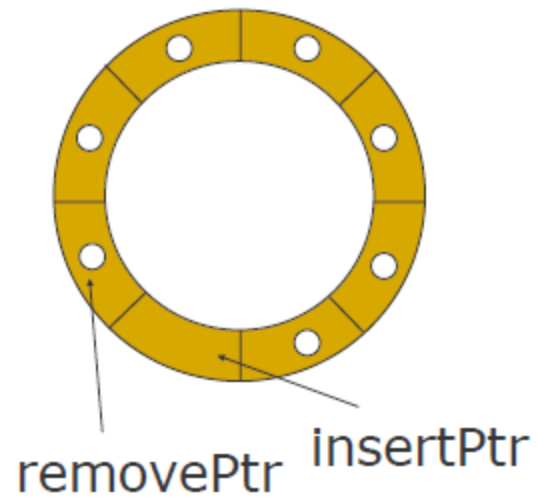


PROBLEMA PRODUCĂTOR – CONSUMATOR

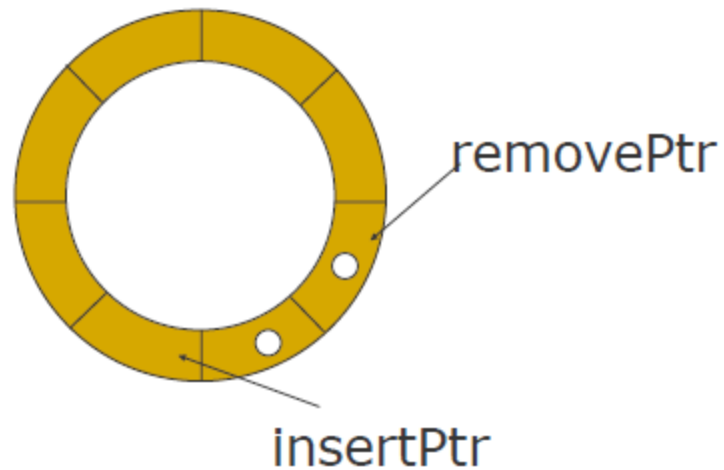
**BUFFER FULL:
Producer must
be blocked!**



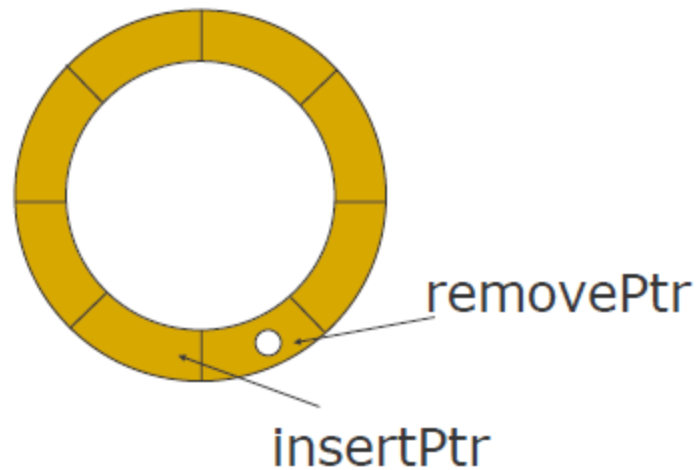
PROBLEMA PRODUCĂTOR – CONSUMATOR



PROBLEMA PRODUCĂTOR – CONSUMATOR

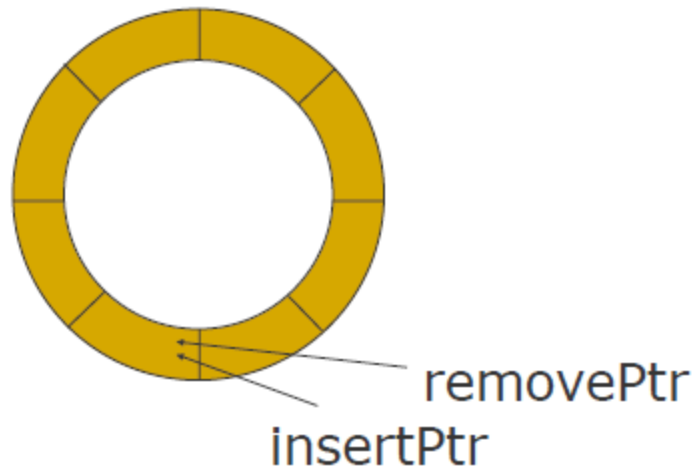


PROBLEMA PRODUCĂTOR – CONSUMATOR



PROBLEMA PRODUCĂTOR – CONSUMATOR

**BUFFER EMPTY:
Consumer must
be blocked!**



PROBLEMA PRODUCĂTOR – CONSUMATOR

- Abordare fără sincronizare

Producător

```
// insert item  
buffer[ insertPtr ] = data;  
insertPtr = (insertPtr+1)%N;
```

Consumator

```
// remove item  
result = buffer[removePtr];  
removePtr = (removePtr +1)%N;
```

PROBLEMA PRODUCĂTOR – CONSUMATOR

- Două tipuri de procese
 - Producător: produce mesaje (date)
 - Consumator: consumă mesaje (date)
- Buffer cu capacitate limitată
- Condiții de așteptare
 - Producător: bufferul este plin
 - Consumator: bufferul este gol
- Condiție de trezire
 - Producător: este loc în buffer
 - Consumator: sunt mesaje în buffer
- Excludere mutuală pentru modificarea bufferului

PROBLEMA PRODUCĂTOR – CONSUMATOR

○ Soluție

- Prevenirea supraîncărcării bufferului
 - Blocăm producătorii dacă bufferul este plin
 - **Semafor cu contor** (slots)
 - arată numărul de poziții libere (free slots)
 - Dacă 0 -> blochează producătorii
- Prevenirea subîncărcării
 - Blocăm consumatorii dacă bufferul este gol
 - **Semafor cu contor** (items)
 - arată numărul de mesaje din buffer (items)
 - Dacă 0 -> blochează consumatorii
- Protejarea accesului la buffer și la pointeri
 - **Semafor binar** – lacăt (mutex)

PROBLEMA PRODUCĂTOR – CONSUMATOR

○ Soluție

Producător

```
// check if buffer is full
sem_wait(slots);
// get mutual exclusion to buffer
mutex_lock(mutex);
// insert item
buffer[ insertPtr ] = data;
insertPtr = (insertPtr+1)%N;
// release the buffer
mutex_unlock(mutex);
// new permission to consumer
sem_signal(items);
```

Consumer

```
// check if buffer is empty
sem_wait(items);
// get mutual exclusion to buffer
mutex_lock(mutex);
// remove item
result = buffer[removePtr];
removePtr = (removePtr +1)%N;
// release the buffer
mutex_unlock(mutex);
// new permission to producer
sem_signal(slots);
```

○ Ordinea blocării este esențială

- Ex. dacă blocăm mai întâi lacătul pt buffer -> DEADLOCK

VARIABILE DE CONDIȚIE

○ Folosite în contextul excluderii mutuale

- Comunică informații despre starea unei resurse partajate
- Un thread așteaptă îndeplinirea unei condiții pentru a-și continua execuția
 - Se află deja în regiunea critică – are lacătul blocat
 - Așteaptă ca **un alt thread să efectueze o acțiune** care să ducă la îndeplinirea condiției
 - Condiția reflectă starea resursei partajate
- Un alt thread trezește threadul adormit la îndeplinirea condiției

○ Conține

- O coadă de așteptare
- Trei operații atomice
 - **Wait** - Așteptare
 - Threadul apelant este suspendat și pus în coada de așteptare până la recepționarea unui semnal (signal). Pe durata așteptării eliberează lacătul. La revenire din wait se re-blochează lacătul.
 - **Signal** – Semnal
 - Un thread din coada de așteptare este eliberat și trezit. Threadul care apelează signal încă deține lacătul → threadul care se trezește intră în coada de așteptare pentru eliberarea lacătului.
 - **Broadcast**
 - Toate threadurile din coada de așteptare sunt eliminate și trezite

VARIABILE CONDIȚIONALE

- Mecanism de așteptare
- Este **resursă partajată** – trebuie protejată
 - Folosim un lacăt pentru a proteja accesul la var. cond
- Fiecărei variabile de condiție i se asociază un lacăt
 - **Wait** – deblochează automat lacătul și suspendă threadul
 - Alte threaduri care așteaptă după lacăt pot bloca lacătul
 - **Signal** – trezește threadul
 - Blochează automat lacătul înainte de reluarea/trezirea threadului
 - Poate fi pierdut – dacă nu există proces blocat în wait în momentul apariției signal-ului
- Importanța retestării condiției după trezire
 - Când un thread este trezit din așteptare nu este sigur că condiția după care aștepta este încă îndeplinită → trebuie să retesteze condiția

VAR. DE CONDIȚIE VS. SEMAFOARE

Semafoare	Var. de condiție
Valoare întreagă	Nu este o val întreagă
Wait – nu este întotdeauna blocant	Wait – este întotdeauna blocant
Signal – trezește un thread sau incrementează contorul	Signal – trezește un thread sau e pierdut
Dacă signal trezește un thread ambele își pot continua execuția	Dacă signal trezește un thread doar unul își poate continua execuția

PROBLEMA PRODUCĂTOR – CONSUMATOR

- Soluție – cu lacăte și variabile de condiție

Variable globale

```
const N = 100; // number of slots in buffer
int count = 0; // number of messages in buffer
Lock mutex ;    // provides mutual exclusion to buffer
Condition_var producers; // controls producers' access to buffer
Condition_var consumers; // controls consumers' access to buffer
```

PROBLEMA PRODUCĂTOR – CONSUMATOR

○ Soluție – cu lacăte și variabile de condiție

Producător

```
// get mutual exclusion to buffer
mutex_lock(mutex);

// check if buffer is full
while ( count == N)
    producers.wait(mutex);

// insert item
insert_item(item);
count++;

// wake up a consumer
if ( count == 1)
    consumers.signal();

// release the buffer
mutex_unlock(mutex);
```

Consumer

```
// get mutual exclusion to buffer
mutex_lock(mutex);

// check if buffer is empty
while ( count == 0)
    consumers.wait(mutex);

// remove item
item = remove_item();
count--;

// wake up a producer
if ( count == N-1)
    producers.signal();

// release the buffer
mutex_unlock(mutex);
```

PROBLEMA CITITORI - SCRITORI

- Un set de date sunt partajate între mai multe procese
- Două tipuri de procese
 - Cititori – doar citesc datele (nu le modifică)
 - Scriitori – modifică datele
- Reguli
 - Mai mulți cititori pot citi simultan datele
 - La un moment dat doar un scriitor poate modifica datele
 - Dacă un scriitor scrie date, atunci nici un cititor nu are voie să citească

	Reader	Writer
Reader	OK	No
Writer	No	No

Copyright © University of Illinois CS 241 Staff

PROBLEMA CITITORI - SCRITORI

○ Soluția 1: cu semafoare

```
int readerCount
semaphore writer = 1, mutex = 1
```

WRITER:

```
while (true)
{
    wait(writer)
    ...
    write operation
    ...
    signal(writer)
}
```

READER:

```
while (true)
{
    wait(mutex)
    readerCount++
    if(readerCount == 1) wait(writer)
    signal(mutex)

    read operation

    wait(mutex)
    readerCount--
    if(readerCount == 0) signal(writer)
    signal(mutex) }
```

○ cititorii au prioritate

○ Scriitorii pot fi înfometati

Stallings pg. 240

PROBLEMA CITITORI - SCRITORI

○ Soluția 2: Scriitorii au prioritate

- Dacă apare un scriitor nici un cititor nou nu mai primește acces la date

```
int readerCount = 0, writerCount = 0  
semaphore writer=1, reader=1, x=1, y=1, z=1
```

WRITER:

```
while (true)  
{  
    wait(y)  
    writerCount++  
    if(writerCount == 1) wait(reader)  
    signal(y)  
  
    wait(writer)  
    write operation  
    signal(writer)  
  
    wait(y)  
    writerCount--  
    if(writerCount == 0) signal(reader)  
    signal(y)  
}
```

READER:

```
while (true)  
{  
    wait(z)  
    wait(reader)  
    wait(x)  
    readerCount++  
    if(readerCount == 1) wait(writer)  
    signal(x)  
    signal(reader)  
    signal(z)  
  
    read operation  
  
    wait(x)  
    readerCount--  
    if(readerCount == 0) signal(writer)  
    signal(x)  
}
```

PROBLEMA MOLECULE DE APA (H₂O)

- Formarea moleculelor de apă
 - Două tipuri de atomi (threaduri) – O si H
 - Barieră – pt așteptarea îndeplinirii condiției de formare a moleculei

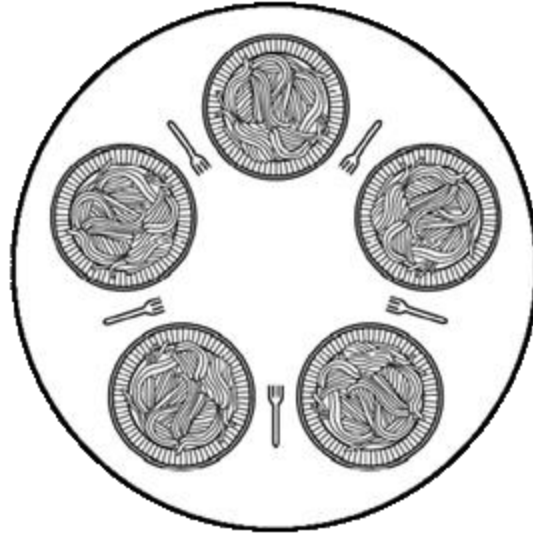
```
Semaphore mutex = 1;  
Semaphore h_wait = 0;  
Semaphore o_wait = 0;  
int count = 0;
```

```
hReady() {  
    P(mutex);  
    count++;  
    if(count %2 == 1) {  
        V(mutex);  
        P(h_wait);  
    } else {  
        V(o_wait);  
        P(h_wait);  
        V(mutex);  
    }  
}
```

```
    return;  
}
```

```
oReady()  
{  
    P(o_wait);  
    V(h_wait);  
    V(h_wait);  
    makeWater();  
  
    return;  
}
```

PROBLEMA FILOZOFILOR



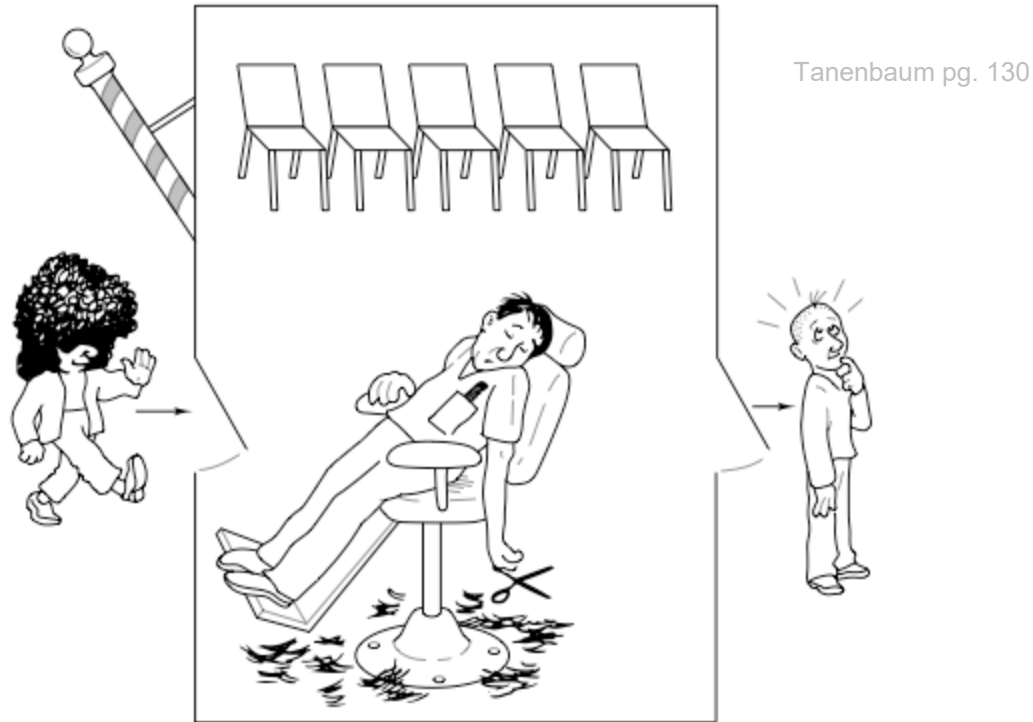
Tanenbaum pg. 125

- 5 filozofi stau așezați la o masă circulară
- Fiecare filozof are în față o farfurie cu spaghetti
- La stânga și la dreapta fiecărei farfurii, se găsește câte un bețișor
- Un filozof are nevoie de două bețișoare pt a putea mânca
- Interblocarea și înfometarea trebuie evitate

PROBLEMA FILOZOFIILOR

- Ciclul de viață al fiecărui filozof
 - perioade de meditație și
 - perioade de hrănire
- Când un filozof este înfometat
 - încearcă să ridice bețele aflate în stânga și în dreapta farfuriei proprii
- Dacă are la îndemână ambele bețe
 - va putea mânca și
 - într-un final va pune bețele înapoi pe masă, în stânga și în dreapta farfuriei
- Va continua, desigur, cu o perioadă de meditație
- Pe scurt
 - Gândește
 - Ridică bățul drept
 - Ridică bățul stâng
 - Mânâncă
 - Așează bățul stâng
 - Așează bățul drept
 - Repetă

PROBLEMA FRIZERULUI SOMNOROS



- Într-o frizerie sunt N scaune de așteptare și un scaun special pe care stă clientul să fie tuns de singurul frizer care lucrează în frizerie
- Dacă nu sunt clienți, Frizerul doarme
- Când un Client intră în frizerie
 - Dacă toate scaunele de așteptare sunt ocupate, atunci Clientul părăsește frizeria.
 - Dacă sunt scaune libere și Frizerul este ocupat, atunci Clientul se așează pe un scaun și așteaptă.
 - Dacă Frizerul doarme, atunci Clientul îl trezește.